INTERPRETABILIDADE DE MODELOS DE ML

SHAP, LIME, Feature Importance e Muito Mais — Nivel Especialista

Breno Ferrari · Preparacao Claro · Abril 2026

#	Secao
1	Por que Interpretabilidade Importa
2	Taxonomia: Interpretabilidade vs Explicabilidade
3	Metodos Globais — Visao do Modelo Completo
4	Feature Importance por Impureza (Tree-based)
5	Permutation Importance
6	Partial Dependence Plot (PDP) e ICE
7	SHAP — SHapley Additive exPlanations
8	LIME — Local Interpretable Model-agnostic Explanations
9	Outros Metodos Relevantes
10	Comparacao dos Metodos
11	Interpretabilidade em Producao
12	Aplicacoes em Telecom e Seguros
13	Checklist do Especialista

1 · POR QUE INTERPRETABILIDADE IMPORTA

Modelos de ML sao cada vez mais usados em decisoes que afetam pessoas e negocios. Um modelo que voce nao consegue explicar e um modelo que voce nao consegue confiar, auditar ou melhorar. Interpretabilidade nao e acessorio -- e parte do ciclo de desenvolvimento.

1.1 Os Quatro Pilares da Necessidade

Pilar	Descricao	Exemplo em Telecom
Confianca	Entender por que o modelo toma determinada decisao e necessario para adocao	Modelo de churn recomenda retencao para cliente que ja cancelou -- bug
Regulatorio	LGPD e regulacoes setoriais exigem explicabilidade em decisoes automatizadas	Cliente questiona por que credito negado -- empresa precisa expli
Debugging	Identificar e corrigir erros sistematicos no modelo	SHAP revela que modelo usa codigo de area como proxy de renda -- c
Geracao de valor	Insights de interpretabilidade viram conhecimento de negocio	Feature importance revela que reclamacoes em canal X prediz churn m

1.2 O Trade-off Interpretabilidade vs Performance

Existe uma tensao real entre modelos interpretaveis por design e modelos de alta performance. Nao e uma regra absoluta -- SHAP e outros metodos post-hoc permitem explicar modelos complexos -- mas e importante entender o espectro:

Modelo	Interpretabilidade	Performance Tipica	Quando Escolher
Regressao Logistica	Alta (coeficientes diretos)	Boa em dados lineares	Quando explicabilidade e requisito regulatorio
Decision Tree	Alta (regras visuais)	Moderada (overfitting)	Quando regras de negocio sao necessarias
Random Forest	Media (feature importance)	Alta	Baseline robusto com explicabilidade parcial
XGBoost/LightGBM	Baixa por design	Muito Alta	Producao com SHAP para explicacao post-hoc
Redes Neurais	Muito Baixa (black box)	Muito Alta em dados nao-estruturados	Imagem, texto, audio -- com LIME/SHAP
Modelos Lineares + Splines	Alta	Boa	GAMs -- alternativa interpretavel ao boosting

2 · TAXONOMIA: INTERPRETABILIDADE vs EXPLICABILIDADE

Os termos sao usados de forma intercambiavel no mercado, mas ha distincoes tecnicas importantes. Conhecer a taxonomia permite escolher a ferramenta certa para cada situacao.

2.1 Dimensoes de Classificacao

Dimensao	Opcao A	Opcao B
Escopo	Global -- explica o modelo como um todo	Local -- explica uma predicao especifica
Dependencia do modelo	Model-specific -- exclusivo para um tipo de modelo	Model-agnostic -- funciona para qualquer modelo
Timing	Intrinseca -- modelo e interpretavel por design	Post-hoc -- explicacao aplicada apos o treino
Tipo de saida	Feature importance -- ranking de importancia	Contrafactual -- o que mudaria a predicao?

2.2 Mapa dos Metodos

Metodo	Escopo	Model-agnostic?	Tipo
Coeficientes (Reg. Logistica)	Global	Nao	Intrinseca
Feature Importance (Impureza)	Global	Nao (tree-based)	Post-hoc
Permutation Importance	Global	Sim	Post-hoc
PDP / ICE	Global	Sim	Post-hoc
SHAP Global	Global	Sim	Post-hoc
SHAP Local	Local	Sim	Post-hoc
LIME	Local	Sim	Post-hoc
Counterfactual Explanations	Local	Sim	Post-hoc
Anchors	Local	Sim	Post-hoc
GAM (Generalized Additive Models)	Global	Nao	Intrinseca
Decision Rules	Global/Local	Nao	Intrinseca

3 · METODOS GLOBAIS -- VISAO DO MODELO COMPLETO

Metodos globais explicam o comportamento geral do modelo -- quais features mais importam, como cada feature afeta a predicao em media, e qual e o comportamento sistematico do modelo na base toda.

3.1 Coeficientes em Modelos Lineares

O que e: em regressao logistica e linear, os coeficientes sao diretamente interpretaveis. Cada coeficiente representa o efeito marginal da feature sobre a saida, mantendo as demais constantes.

- **Regressao Linear:** beta_i = variacao em Y para aumento de 1 unidade em X_i.
- **Regressao Logistica:** exp(beta_i) = Odds Ratio. exp(0.7) = 2.01 significa que a feature duplica as chances do evento.
- **Coeficientes padronizados:** padronizar X antes de treinar permite comparar importancia relativa das features diretamente.
- **p-valor:** testa H0: beta = 0. p < 0.05 = significativamente diferente de zero.
- **Intervalo de confianca:** se o IC do coeficiente inclui zero, a feature pode nao ser relevante.

```
import pandas as pd from sklearn.linear_model import LogisticRegression model =
LogisticRegression(max_iter=1000).fit(X_train, y_train) # Coeficientes e Odds Ratios coef_df =
pd.DataFrame({ "feature": feature_names, "coef": model.coef_[0], "odds_ratio":
np.exp(model.coef_[0]) }).sort_values("odds_ratio", ascending=False) print(coef_df)
```

Limitacao critica: coeficientes sao confiaveis apenas se as premissas do modelo sao satisfeitas (linearidade, sem multicolinearidade, etc.). Com multicolinearidade, coeficientes ficam instáveis e perdem interpretabilidade direta.

4 · FEATURE IMPORTANCE POR IMPUREZA (TREE-BASED)

Feature importance baseada em impureza e o metodo nativo de Random Forest, XGBoost, LightGBM e CatBoost. Mede quanto cada feature reduz a impureza (Gini ou Entropia em classificacao, MSE em regressao) ao longo de todas as arvores.

4.1 Como Funciona

- **Para cada arvore:** somar a reducao de impureza em todos os nos onde a feature foi usada para split, ponderada pelo numero de amostras.
- **Para o ensemble:** media das importancias de todas as arvores.
- **Normalizacao:** importancias somam 1 (ou 100%) -- sao relativas entre si.
- **Ganho (XGBoost):** media do ganho de impureza de todos os splits da feature. Mais estavel que frequencia.
- **Cobertura (XGBoost):** numero de amostras afetadas pelos splits da feature.
- **Frequencia (XGBoost):** quantas vezes a feature foi usada como split -- pode favorecer features continuas.

4.2 Codigo e Visualizacao

```
import pandas as pd import matplotlib.pyplot as plt from sklearn.ensemble import
RandomForestClassifier import xgboost as xgb # Random Forest rf =
RandomForestClassifier(n_estimators=200, random_state=42) rf.fit(X_train, y_train) fi_rf =
pd.Series(rf.feature_importances_, index=feature_names) fi_rf.sort_values().plot(kind="barh",
figsize=(10, 8)) plt.title("Feature Importance -- Random Forest (Impureza)") # XGBoost -- tipo de
importancia xgb_model = xgb.XGBClassifier(random_state=42).fit(X_train, y_train) fi_xgb = pd.Series(
xgb_model.get_booster().get_score(importance_type="gain"), name="gain" ) # gain: mais estavel que
weight (frequencia)
```

4.3 Limitacoes Criticas

Esta e a limitacao mais importante de conhecer:

• **Vies para features de alta cardinalidade:** features continuas ou com muitas categorias tem mais oportunidades de split e tendem a ter importancia inflada artificialmente.

• **Nao distingue direcao do efeito:** feature importante pode ter efeito positivo ou negativo -- importancia nao diz qual.

• **Correlacao entre features:** com duas features altamente correlacionadas, a importancia e dividida de forma instavel entre elas. Remova uma ou use Permutation Importance.

• **Nao e importancia causal:** feature importante para o modelo nao significa que e causa do evento -- pode ser correlacionada com uma causa real.

• **Nao e substituivel por SHAP:** Feature Importance e global e nao revela comportamento por observacao individual.

Regra pratica: use Feature Importance para uma primeira triagem rapida. Para decisoes importantes de negocio, sempre complemente com SHAP ou Permutation Importance.

5 · PERMUTATION IMPORTANCE

Permutation Importance e model-agnostic, mais confiavel que importancia por impureza, e resolve o vies de cardinalidade. Ideia central: se uma feature e importante, embaralha-la deve degradar a performance do modelo. Se a performance nao muda, a feature e irrelevante.

5.1 Algoritmo

1. Treinar o modelo e calcular a metrica baseline no conjunto de validacao.
2. Para cada feature j:
3. a. Embaralhar aleatoriamente os valores da feature j (quebrar a relacao com Y).
4. b. Calcular a metrica do modelo com a feature embaralhada.
5. c. Importancia_j = metrica_baseline - metrica_embaralhada.
6. Repetir N vezes para calcular media e desvio (variabilidade da estimativa).
7. Ordenar features por importancia media decrescente.

5.2 Vantagens sobre Feature Importance por Impureza

Aspecto	Impureza (MDI)	Permutation Importance
Vies de cardinalidade	Alto -- favorece features continuas	Nenhum
Model-agnostic	Nao -- apenas tree-based	Sim -- qualquer modelo
Avaliacao em dados de teste	Nao -- calculado no treino	Sim -- pode usar validacao
Detecta features inuteis	Nao confiavel	Sim -- importancia proximo a zero
Custo computacional	Baixo (calculado no treino)	Medio (N * n_features predicoes extras)
Considera correlacao	Nao	Parcialmente -- features correlacionadas podem ter importancia dil

5.3 Codigo

```
from sklearn.inspection import permutation_importance import pandas as pd # Model-agnostic: funciona com qualquer modelo result = permutation_importance( model, X_val, y_val, n_repeats=30, # repeticoes para estabilidade random_state=42, scoring="roc_auc" # metrica de interesse ) perm_df = pd.DataFrame({ "feature": feature_names, "importance": result.importances_mean, "std": result.importances_std }).sort_values("importance", ascending=False) # Visualizacao com intervalo de confianca perm_df.plot(x="feature", y="importance", kind="barh", xerr="std", figsize=(10,8)) plt.axvline(0, color="red", linestyle="--") # linha zero plt.title("Permutation Importance +/- Std")
```

Atencao: features com importancia negativa significam que embaralha-las MELHORA a performance -- a feature esta introduzindo ruido ou e um artefato. Candidata a remocao.

6 · PARTIAL DEPENDENCE PLOT (PDP) E INDIVIDUAL CONDITIONAL EXPECTATION (ICE)

PDP e ICE revelam COMO cada feature afeta a predicao -- nao apenas quanto. Eles mostram a direcao e a forma da relacao entre uma feature e o target, marginalizando o efeito das demais features.

6.1 Partial Dependence Plot (PDP)

O que e: para cada valor possivel de uma feature X_j, calcula a predicao media do modelo mantendo X_j fixo e variando todas as outras features com seus valores reais da base. O resultado e a relacao marginal media entre X_j e Y.

Formula conceitual: PDP(x_j) = (1/n) * soma[f(x_j, x_-j^i)] para i=1..n

- **Interpretacao:** o eixo X e o valor da feature; o eixo Y e a predicao media do modelo.
- **Relacao linear crescente:** feature tem efeito positivo monotono.
- **Curva em U:** feature tem efeito nao-monotono -- otimo no meio do range.
- **Linha plana:** feature nao tem efeito sobre a predicao (feature inutill).
- **Limitacao:** assume independencia entre features -- em dados correlacionados, pode gerar combinacoes de valores imposssiveis.

6.2 Individual Conditional Expectation (ICE)

O que e: versao individualizada do PDP. Em vez de calcular a media, plota uma linha para CADA observacao da base. O PDP e a media de todas as linhas ICE.

Por que ICE e superior ao PDP: quando as linhas ICE divergem entre si, ha interacao entre a feature e outras variaveis -- o efeito e heterogeneo. O PDP mostra apenas a media e esconde essa heterogeneidade.

- **Linhas paralelas:** efeito homogeneo -- todos os clientes respondem igual a variacao da feature.
- **Linhas que se cruzam:** interacao forte -- o efeito depende de outras features.
- **ICE centrado (c-ICE):** subtrair o valor no ponto de referencia de cada linha. Facilita comparacao da direcao do efeito entre observacoes.

6.3 Codigo PDP e ICE

```
from sklearn.inspection import PartialDependenceDisplay import matplotlib.pyplot as plt # PDP simples -- uma feature fig, ax = plt.subplots(figsize=(8, 5)) PartialDependenceDisplay.from_estimator( model, X_train, features=["arpu"], # feature de interesse kind="average", # PDP (media) ax=ax ) # ICE -- linhas individuais fig, ax = plt.subplots(figsize=(10, 6)) PartialDependenceDisplay.from_estimator( model, X_train, features=["arpu"], kind="individual", # ICE subsample=200, # amostrar para nao poluir alpha=0.1, # transparencia ax=ax ) # ICE + PDP sobrepostos (melhor opcao) PartialDependenceDisplay.from_estimator( model, X_train, features=["arpu", "consumo_dados"], # multiplas features kind="both", # ICE + media subsample=300, alpha=0.05 ) # PDP 2D -- interacao entre duas features PartialDependenceDisplay.from_estimator( model, X_train, features=[("arpu", "tempo_contrato")], # par de features kind="average" )
```

7 · SHAP -- SHapley Additive exPlanations

SHAP e o metodo mais completo, matematicamente fundamentado e amplamente adotado de interpretabilidade de modelos de ML. Baseia-se na teoria dos jogos cooperativos (Shapley Values de Lloyd Shapley, Premio Nobel de Economia 2012) e fornece explicacoes tanto globais quanto locais de forma consistente.

7.1 A Intuicao por Tras do SHAP

Imagine que cada feature e um "jogador" numa equipe. O "premio" e a diferenca entre a predicao para uma observacao especifica e a predicao media do modelo (baseline). O SHAP distribui esse premio de forma justa entre os jogadores (features) -- proporcional a contribuicao de cada um.

Formalmente: o valor SHAP de uma feature j para uma observacao x e a contribuicao marginal media de j considerando todas as possiveis ordens de inclusao das features.

> **Propriedade fundamental:** soma dos valores SHAP de todas as features = predicao do modelo - predicao media do modelo (baseline).

> **Em termos matematicos:** f(x) = baseline + SHAP_1 + SHAP_2 + ... + SHAP_n Onde baseline = E[f(X)] (predicao media na base de treino)

7.2 Propriedades Axiomaticas do SHAP

O SHAP e o unico metodo que satisfaz simultaneamente as 4 propriedades desejadas de qualquer explicacao justa:

Propriedade	Significado	Por que Importa
Eficiencia	Soma dos SHAP = predicao - baseline	Explicacao e completa e consistente
Simetria	Features com mesma contribuicao tem mesmo SHAP	Justica entre features
Dummy	Feature sem efeito tem SHAP = 0	Features irrelevantes nao poluem a explicacao
Monotonicidade	Feature com maior contribuicao em todos os contextos tem maior SHAP	SHAP reflete logica do modelo

7.3 Tipos de Explainers SHAP

Explainer	Modelos Suportados	Complexidade	Quando Usar
TreeExplainer	XGBoost, LightGBM, CatBoost, RF, Extra Trees	O(TLD^2) -- rapido	Sempre para tree-based -- mais rapido e exato
LinearExplainer	Regressao Linear, Logistica, Lasso, Ridge	O(n) -- muito rapido	Modelos lineares -- SHAP = coeficiente * valor
DeepExplainer	Redes Neurais (TensorFlow, PyTorch)	Alto	Deep Learning -- aproximacao por backprop
GradientExplainer	Redes Neurais com gradientes	Alto	Alternativa ao DeepExplainer
KernelExplainer	Qualquer modelo (black box)	Muito Alto -- O(2^n) features	Ultimo recurso -- lento, use amostra

7.4 SHAP para Tree-based Models -- Codigo Completo

```
import shap import matplotlib.pyplot as plt # 1. CRIAR EXPLAINER explainer =
shap.TreeExplainer(xgb_model) # ou rf, lgbm, catboost # 2. CALCULAR SHAP VALUES shap_values =
explainer.shap_values(X_test) # shap_values: array (n_samples, n_features) # Para classificacao
binaria: shap_values[1] = classe positiva # Alternativa com objeto Explanation (mais moderno)
explanation = explainer(X_test) # explanation.values: SHAP values # explanation.base_values:
baseline por observacao # explanation.data: valores reais das features
```

7.5 Visualizacoes SHAP Globais

Summary Plot (Beeswarm) -- O mais importante:

Mostra a distribuicao dos SHAP values de cada feature para todas as observacoes. Eixo X = valor SHAP (contribuicao). Cor = valor real da feature (azul=baixo, vermelho=alto). Eixo Y = features ordenadas por importancia media absoluta.

```
# Beeswarm plot -- distribuicao dos SHAP values shap.summary_plot(shap_values, X_test, feature_names=feature_names) # Bar plot -- importancia global (media do |SHAP|) shap.summary_plot(shap_values, X_test, plot_type="bar", feature_names=feature_names)
```

> **Como interpretar o Beeswarm:** - Feature no topo = mais importante globalmente - Pontos vermelhos a direita: valor alto da feature aumenta a predicao - Pontos azuis a direita: valor baixo da feature aumenta a predicao - Distribuicao ampla: efeito heterogeneo -- depende do contexto - Distribuicao concentrada: efeito homogeneo -- mesmo efeito para todos

SHAP Dependence Plot -- Relacao feature x SHAP:

Equivalente ao PDP mas usando SHAP. Eixo X = valor da feature, Eixo Y = SHAP value. Cor = segunda feature com maior interacao automaticamente detectada.

```
# Dependence plot -- relacao feature x SHAP value shap.dependence_plot( "arpu", # feature de interesse shap_values, X_test, interaction_index="consumo_dados" # colorir por interacao # interaction_index="auto" detecta automaticamente )
```

SHAP Heatmap -- Visao completa:

Heatmap onde linhas = features, colunas = observacoes, cor = SHAP value. Permite identificar padroes sistematicos de explicacao.

```
shap.plots.heatmap(explanation[:500]) # amostrar para legibilidade
```

7.6 Visualizacoes SHAP Locais

Waterfall Plot -- Explicacao de UMA predicao:

Mostra como cada feature empurra a predicao para cima ou para baixo a partir do baseline. A explicacao mais intuitiva para apresentar a stakeholders nao-tecnicos.

```
# Waterfall -- explicar UMA observacao idx = 0 # indice do cliente a explicar shap.plots.waterfall(explanation[idx]) # Visualizacao legado (ainda muito usada) shap.force_plot( explainer.expected_value, shap_values[idx], X_test.iloc[idx], matplotlib=True )
```

> **Como interpretar o Waterfall:** - Base value: predicao media do modelo (E[f(X)]) - Barras vermelhas: features que aumentam a predicao - Barras azuis: features que diminuem a predicao - f(x): predicao final para esta observacao - A soma de todas as barras + base value = f(x)

Force Plot -- Multiplas observacoes:

Versao interativa do Waterfall para multiplas observacoes simultaneamente. Permite identificar grupos de observacoes com padroes de explicacao similares.

```
# Force plot interativo para multiplas observacoes shap.force_plot( explainer.expected_value, shap_values[:100], X_test.iloc[:100] ) # Gera HTML interativo -- usar em Jupyter
```

7.7 SHAP Interaction Values

SHAP pode decompor os valores em efeitos principais e efeitos de interacao entre pares de features. Revela sinergias e antagonismos entre features.

```
# SHAP Interaction Values -- apenas TreeExplainer shap_interact = explainer.shap_interaction_values(X_test) # shape: (n_samples, n_features, n_features) # diagonal: efeito principal de cada feature # fora da diagonal: efeito de interacao entre pares shap.summary_plot(shap_interact, X_test) # interaction summary
```

7.8 SHAP como Feature Selection

SHAP values podem ser usados para selecionar features de forma mais robusta que Feature Importance por impureza:

```
# Importancia global: media dos |SHAP values| mean_abs_shap = pd.Series(
np.abs(shap_values).mean(axis=0), index=feature_names ).sort_values(ascending=False) # Remover
features com SHAP medio < threshold threshold = 0.001 features_selected =
mean_abs_shap[mean_abs_shap > threshold].index.tolist() print(f"Features selecionadas:
{len(features_selected)}")
```

8 · LIME -- Local Interpretable Model-agnostic Explanations

LIME explica predicoes individuais treinando um modelo simples e interpretavel (regressao linear ou arvore rasa) localmente -- na vizinhanca da observacao de interesse. O modelo local aproxima o comportamento do modelo complexo apenas naquele ponto.

8.1 Como Funciona

1. Selecionar a observacao x que se quer explicar.
2. Gerar N amostras perturbadas na vizinhanca de x (ruido aleatorio nas features).
3. Calcular a predicao do modelo complexo para cada amostra perturbada.
4. Pesar as amostras pela proximidade a x (amostras proximas tem maior peso).
5. Treinar um modelo linear nas amostras perturbadas ponderadas.
6. Os coeficientes do modelo linear sao a explicacao local -- quais features mais influenciaram ESTA predicao.

8.2 Codigo LIME

```
from lime import lime_tabular # Criar explainer lime_explainer = lime_tabular.LimeTabularExplainer( X_train.values, feature_names=feature_names, class_names=["nao_churn", "churn"], mode="classification", discretize_continuous=True # discretiza features continuas ) # Explicar UMA observacao idx = 42 exp = lime_explainer.explain_instance( X_test.values[idx], model.predict_proba, # funcao de predicao num_features=10, # top N features na explicacao num_samples=5000 # amostras para treinar o modelo local ) # Visualizacoes exp.show_in_notebook() # interativo em Jupyter exp.as_pyplot_figure() # matplotlib estatico exp.as_list() # lista de (feature, coeficiente)
```

8.3 LIME vs SHAP -- Quando Usar Cada Um

Aspecto	SHAP	LIME
Base teorica	Teoria dos jogos (Shapley) -- axiomaticamente justo	Aproximacao local -- sem garantias globais
Consistencia	Global e local consistentes entre si	Explicacoes locais podem ser instaveis
Velocidade (tree-based)	Rapido (TreeExplainer)	Lento (N amostras por observacao)
Model-agnostic	Sim (KernelExplainer -- lento)	Sim -- qualquer modelo
Estabilidade	Alta -- mesma observacao, mesmo resultado	Media -- depende das amostras aleatorias
Interacoes	Captura via SHAP Interaction Values	Nao captura diretamente
Ideal para	Producao, auditoria, feature selection	Exploracao rapida, modelos nao-tabulares
Melhor em	Dados tabulares com tree-based models	Texto e imagem (variantes especializadas)

Recomendacao: para dados tabulares com modelos de boosting (XGBoost, LightGBM), SHAP com TreeExplainer e superior em todos os aspectos. LIME e util quando o modelo e verdadeiramente black box sem suporte de TreeExplainer, ou para dados de texto e imagem.

9 · OUTROS METODOS RELEVANTES

9.1 Counterfactual Explanations

O que e: responde a pergunta "o que precisaria mudar para a predicao ser diferente?". Em vez de explicar por que o modelo tomou uma decisao, mostra o menor conjunto de mudancas nas features para mudar o resultado.

- **Aplicacao:** cliente com churn score alto -- o que ele precisaria mudar para sair do risco?
- **Vantagem:** diretamente acionavel para o cliente e para o negocio.
- **Metodo:** DiCE (Diverse Counterfactual Explanations) -- gera multiplos contrafactuais diversos.
- **Restricoes:** features imutaveis (idade, historico) nao podem ser alteradas no contrafactual.

```
import dice_ml data = dice_ml.Data(dataframe=df_train, continuous_features=num_cols, outcome_name="churn") m = dice_ml.Model(model=xgb_model, backend="sklearn") exp = dice_ml.Dice(data, m) # Gerar contrafactuais para um cliente cf = exp.generate_counterfactuals( query_instances=X_test.iloc[[42]], total_CFs=3, # 3 alternativas desired_class="opposite", # mudar a classe features_to_vary=["arpu", "consumo_dados", "reclamacoes"] ) cf.visualize_as_dataframe()
```

9.2 Anchors

O que e: regras do tipo if-then que "ancoram" a predicao. Uma ancora e uma regra simples que, quando satisfeita, garante a mesma predicao com alta precisao. "SE arpu < 50 E reclamacoes > 3, ENTAO churn com 92% de precisao."

- **Vantagem:** extremamente intuitivo e direto para stakeholders de negocio.
- **Cobertura:** qual proporcao da base a ancora se aplica.
- **Precisao:** em que proporcao das vezes a ancora leva a mesma predicao.
- **Desvantagem:** ancoras podem ser muito especificas (baixa cobertura) ou muito vagas (baixa precisao).

9.3 GAM -- Generalized Additive Models

O que e: alternativa interpretavel por design ao boosting. Modela a saida como soma de funcoes suaves de cada feature -- similar a regressao linear mas sem assumir linearidade.

Formula: g(E[Y]) = beta_0 + f_1(X_1) + f_2(X_2) + ... + f_n(X_n)

- **Interpretabilidade:** cada f_i pode ser plotada -- voce ve exatamente como a feature afeta a saida.
- **Performance:** geralmente inferior ao XGBoost, mas superior a regressao logistica.
- **Quando usar:** quando interpretabilidade regulatoria e obrigatoria mas modelo linear nao tem performance suficiente.
- **Biblioteca:** interpret-ml (Microsoft) ou pygam.

```
from interpret.glassbox import ExplainableBoostingClassifier # GAM com interacoes (EBM -- Explainable Boosting Machine) ebm = ExplainableBoostingClassifier(random_state=42) ebm.fit(X_train, y_train) # Explicacao global from interpret import show ebm_global = ebm.explain_global() show(ebm_global) # Explicacao local ebm_local = ebm.explain_local(X_test[:10], y_test[:10]) show(ebm_local)
```

9.4 TCAV -- Testing with Concept Activation Vectors

O que e: metodo para redes neurais que explica predicoes em termos de conceitos de alto nivel definidos pelo usuario (ex: "cliente inadimplente", "plano premium") ao inves de features individuais. Especialmente util em visao computacional.

Aplicacao em telecom: explicar quais conceitos de negocio mais influenciam um modelo de NLP treinado em textos de reclamacoes de clientes.

10 · COMPARACAO DOS METODOS

Metodo	Escopo	Agnostico	Velocidade	Estabilidade	Melhor Para
Coeficientes	Global	Nao	Rapido	Alta	Modelos lineares -- auditoria regulatoria
MDI (Impureza)	Global	Nao	Rapido	Media	Triagem rapida de features
Permutation Imp.	Global	Sim	Medio	Alta	Feature selection confiavel
PDP	Global	Sim	Medio	Alta	Entender direcao e forma do efeito
ICE	Global	Sim	Medio	Alta	Detectar heterogeneidade e interacoes
SHAP Global	Global	Sim	Rapido*	Alta	Feature importance + direcao do efeito
SHAP Local	Local	Sim	Rapido*	Alta	Explicar predicoes individuais
SHAP Interact.	Global/Local	Nao	Lento	Alta	Detectar interacoes entre features
LIME	Local	Sim	Lento	Media	Black box, texto, imagem
Counterfactual	Local	Sim	Medio	Media	Recomendacoes acionaveis ao usuario
Anchors	Local	Sim	Lento	Media	Regras simples para stakeholders
GAM/EBM	Global	Nao	Rapido	Alta	Interpretabilidade intrinseca com boa perf.

* Rapido com TreeExplainer. KernelExplainer (model-agnostic) e muito mais lento.

Protocolo recomendado para dados tabulares com tree-based models: 1. Feature Importance (MDI) -- triagem inicial rapida 2. Permutation Importance -- validar MDI e confirmar features relevantes 3. SHAP Global (Beeswarm + Bar) -- importancia + direcao do efeito 4. SHAP Dependence Plot -- relacao feature x SHAP por observacao 5. PDP/ICE -- confirmar forma do efeito e heterogeneidade 6. SHAP Waterfall -- explicar predicoes individuais criticas

11 · INTERPRETABILIDADE EM PRODUCAO

Interpretabilidade nao e apenas uma ferramenta de analise -- em producao, ela e parte do pipeline de monitoramento, debugging e comunicacao com o negocio.

11.1 SHAP em Producao -- Score por Cliente

Em modelos de churn, propensao e fraud, e pratica comum gerar nao apenas o score, mas tambem as top N features que mais contribuiram para aquele score especifico.

```
def score_com_explicacao(model, explainer, X_new, n_features=5): """ Retorna score + top features que mais contribuiram para cada observacao. """ # Score scores = model.predict_proba(X_new)[:, 1] # SHAP values shap_vals = explainer.shap_values(X_new) resultados = [] for i in range(len(X_new)): # Top N features por |SHAP| shap_i = pd.Series(shap_vals[i], index=feature_names) top = shap_i.abs().sort_values(ascending=False).head(n_features) resultados.append({ "score_churn": scores[i], "top_features": top.index.tolist(), "top_shap": top.values.tolist(), "direcao": shap_i[top.index].apply( lambda v: "aumenta" if v > 0 else "diminui" ).tolist() }) return pd.DataFrame(resultados)
```

11.2 Monitoramento de SHAP ao Longo do Tempo

SHAP values em producao revelam se o modelo continua explicando o problema da mesma forma -- ou se o comportamento mudou.

- **SHAP mean por feature ao longo do tempo:** se a importancia de uma feature muda drasticamente entre periodos, algo mudou nos dados ou no comportamento.
- **Distribuicao dos SHAP values:** se a distribuicao de SHAP de uma feature muda de forma, o modelo aprendeu um padrao diferente.
- **Top feature mais frequente por cluster de score:** monitorar se as razoes dos scores altos/baixos sao consistentes com o esperado.
- **Alert:** feature que nunca aparecia no top 5 comeca a aparecer -- investigar se e um novo padrao real ou artefato de dado.

11.3 Interpretabilidade para Auditoria

Para modelos sujeitos a auditoria (credito, seguros, RH), interpretabilidade precisa ser documentada e reproduzivel.

- **Documentar baseline:** E[f(X)] do modelo em producao. Se o baseline muda, os SHAP values mudam com ele.
- **Versionar explicacoes:** guardar SHAP values de amostras representativas junto com cada versao do modelo.
- **Teste de disparate impacto:** verificar se SHAP values de features sensiveis (regiao, codigo de area) sao proximos de zero -- modelo nao deve usar proxies de atributos protegidos.
- **Reproducibilidade:** fixar random_state em todos os componentes do pipeline.

11.4 Comunicando SHAP para Stakeholders Nao-Tecnicos

A linguagem do SHAP nao e natural para gerentes e diretores. Traduza os valores SHAP para linguagem de negocio:

```
# Traducao de SHAP para linguagem de negocio def explicacao_negocio(shap_vals, feature_names, X_obs, threshold=0.01): frases = [] shap_s = pd.Series(shap_vals, index=feature_names) for feat, val in shap_s.abs().sort_values(ascending=False).items(): if val < threshold: break feat_val = X_obs[feat] direcao = "aumenta" if shap_s[feat] > 0 else "diminui" frases.append( f"- {feat} = {feat_val:.1f}: {direcao} o risco de churn " f"(impacto: {shap_s[feat]:+.3f})" ) return " ".join(frases)
```

12 · APLICACOES EM TELECOM E SEGUROS

Interpretabilidade nao e abstrata -- ela tem aplicacoes concretas e de alto impacto nos contextos que voce ja conhece.

12.1 Churn -- Por que Este Cliente Vai Sair?

Problema sem interpretabilidade: modelo gera lista de clientes em risco. Time de retencao liga para todos e oferece desconto indiscriminado.

Com SHAP: para cada cliente em risco, sabe-se POR QUE ele esta em risco.

- Cliente A: principal driver e reclamacoes nao resolvidas -- acao: escalada de suporte prioritaria.
- Cliente B: principal driver e queda de consumo de dados -- acao: oferta de upgrade de franquia.
- Cliente C: principal driver e preco relativo ao concorrente -- acao: oferta de desconto.
- Cliente D: principal driver e vencimento proximo de contrato -- acao: campanha de renovacao.

Resultado: acoes personalizadas por driver de churn, nao desconto generico para todos. Maior retencao, menor custo de retencao.

12.2 Precificacao -- O que Justifica Este Preco?

- **Modelo de elasticidade + SHAP:** entender quais features mais impactam a sensibilidade a preco por segmento.
- **PDP de preco:** visualizar como a variacao de preco afeta a probabilidade de conversao para cada segmento de cliente.
- **ICE de preco:** identificar se clientes de diferentes perfis tem curvas de elasticidade distintas.
- **Comunicacao executiva:** "Clientes com ARPU > R$100 e contrato > 24 meses sao 3x menos sensiveis a preco que a media" -- derivado de SHAP dependence plot.

12.3 Fraude -- Por que Este Evento e Suspeito?

- **SHAP waterfall por alerta:** cada evento flagrado como fraude tem uma explicacao individual -- quais features geraram o alerta.
- **Utilidade operacional:** analista de fraude nao precisa investigar o modelo -- investiga o cliente guiado pelas features mais relevantes do SHAP.
- **Auditoria:** em caso de questionamento, a explicacao do alerta e documentada e rastreavel.
- **Deteccao de vies:** se SHAP revela que "regiao" e sempre o principal driver, investigar se e discriminacao geografica ou sinal real.

12.4 Segmentacao -- O que Define Cada Cluster?

Combinando clusterizacao com SHAP: treinar um modelo de classificacao (cluster como variavel Y) e aplicar SHAP para entender quais features definem cada cluster.

```
# Treinar classificador de cluster from sklearn.ensemble import GradientBoostingClassifier clf_cluster = GradientBoostingClassifier(random_state=42) clf_cluster.fit(X, df["cluster"]) # cluster como target # SHAP para entender o que define cada cluster explainer_c = shap.TreeExplainer(clf_cluster) shap_c = explainer_c.shap_values(X) # Para cada cluster, ver quais features mais definem sua identidade for cluster_id in range(n_clusters): mask = df["cluster"] == cluster_id mean_shap = np.abs(shap_c[cluster_id][mask]).mean(axis=0) top_features = pd.Series(mean_shap, index=feature_names)\ .sort_values(ascending=False).head(5) print(f"\nCluster {cluster_id}:") print(top_features)
```

13 · CHECKLIST DO ESPECIALISTA

Use antes de qualquer apresentacao de modelo ou deploy em producao. Item nao marcado = explicabilidade incompleta ou em risco.

	Item	Metodo
■	Feature Importance (MDI) calculada e documentada?	MDI
■	Permutation Importance validou MDI (resultados consistentes)?	Perm. Imp.
■	Features com importancia negativa no Permutation investigadas?	Perm. Imp.
■	SHAP Global (Beeswarm) gerado para o modelo final?	SHAP
■	Direcao do efeito de cada feature top-10 identificada?	SHAP
■	SHAP Dependence Plot das features mais importantes gerado?	SHAP
■	PDP/ICE confirma a direcao do SHAP?	PDP/ICE
■	ICE revelou heterogeneidade? Interacoes investigadas?	ICE
■	SHAP Waterfall de pelo menos 3 casos representativos revisado?	SHAP Local
■	Baseline (E[f(X)]) documentado?	SHAP
■	Soma dos SHAP = predicao - baseline verificada?	SHAP
■	Features sensiveis (regiao, idade) tem SHAP proximo de zero?	SHAP/Auditoria
■	Explicacao traduzida para linguagem de negocio?	Comunicacao
■	Pipeline de explicacao persistido junto com o modelo?	Producao
■	Monitoramento de SHAP ao longo do tempo configurado?	Monitoramento
■	Contrafactuais gerados para casos de alto impacto?	Counterfactual
■	Documentacao de interpretabilidade para auditoria preparada?	Auditoria
■	Stakeholders nao-tecnicos conseguem interpretar a explicacao?	Comunicacao